Lec 136 (LPS)

Q.) Longest Palindromic Subsequence

i/p → "bbbab"
o/p → "bbbb"

relative ordering should be same.

approach:-

i/p → string

s1 = string
s2 = reverse. string

s1 — LCS → (LPS) — s2

use dp.

Lec 128 EDIT DISTANCE

i/p str1 = "horse"
str2 = "ros"

o/p = 3

→ return min operations to perform to make str2 = str1.

oprs. •) delete
•) insert
•) replace

	cost
horse	0
rorse	1
rose	2
ros	3

~~left prop~~

horse
i

ros.

insert → rhorse (i → i)
delete → orse (i → i+1)
replace → rorse. (i → i+1)

j → j+1
j → j
j → j+1

rec + mem

```
int solve (string a, string b,
int i, int j, vector < vector <int>
& dp) {

if (i == a.size())
  return b.size() - j;

if (j == b.size())
  return b.size() - i;

if (dp[i][j] != -1)
  return dp[i][j];

int ans = 0;
if (a[i] == b[j]) {
  return solve (a, b, i+1, j+1, dp);
}
else
{
  int insertAns = 1 + solve (a, b,
                  i, j+1, dp);

  int deleteAns = 1 + solve (a, b, i+1, j,
                  dp);

  int replaceAns = 1 + solve (a, b, i+1,
                   j+1, dp);

  ans = min (insertAns, min (deleteAns,
        replaceAns));
}
return dp[i][j] = ans;
}
```

Tabulation

```
int solveTab (string a, string b) {
vector < vector <int>> dp (a.size()+1,
 vector <int> (b.size()+1, 0));

for (int j=0; j < b.size(); j++) {
  dp [a.length()+1] [j] = b.length()-j;
}

for (int i=0; i < a.size(); i++) {
  dp [i] [b.length()] = a.length()-i;
}

for (int i = a.length() -1; i >=0;
     i--) {
  for (int j = b.length()-1; j >=0;
       j--) {
    int ans = 0;
    if (a[i] == b[j]) {
      ans = dp[i+1][j+1];
    } else {
      int insertAns = 1 + dp[i][j+1];
      int deleteAns = 1 + dp[i+1][j];
      int replace = 1 + dp[i+1][j+1];
      ans = min (insertAns, min (
            deleteAns, replace));
    }
    dp[i][j] = ans;
  }
}
return dp[0][0];
}
```

Lec 138 (Lc 85)

Q.) Maximal Rectangle

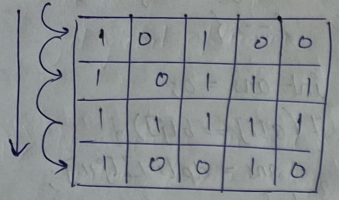
i/p → matrix = {0,1}

1	1	0	0	1	0
1	0	0	1	1	1
0	1	0	1	1	1

o/p = 6

approach Methode

recall "largest area in histogram"



chuck & store area of max rect in histogram by adding (iterating) through each row.

---

code

```
int maxiRect ( vector <vector <int>>
                & matrix) {
int maxi = INT_MIN;
vector <int> histogram (matrix[0].
  size(), 0);
for( int i=0; i<matrix.size(); i++){
  for ( int j=0; j< matrix histogram.
      size(); j++) {
    if (histogram.size (0.
    if (matrix[i][j] == '1') {
      histogram[j] ++;
    }
    else
    { histogram[j] = 0;
    }
  }
  maxi = max(maxi, largestArea
              (histogram));
}
return maxi;
}
}
```

---

lc139 LC = 44

Q.) Wild Card Pattern matching.

a*b?d

? → can take value of any single character.

* → can take any possible combination string.

given: str : abcde
        pattern : a*c?de

↳ invalid (? ≠ " ")

str : aa
p : aa* → valid
(* = " ")

→ Task : Chuck if it is possible to generate str from given pattern

approach

```
      ← i
  a b c d e
  a * c ? e
        ↑
        j
```

```
if(match) {
    fun(str, p, i-1, j-1)
else if ( p[i] == '*')
    f(str, p, i, j-1)
        or
    f(str, p, i-1, j)
else
    return false
```

---

rec + mem

```
bool solve ( string s, string p, int
  i, int j, dp ) {
if ( i<0 && j>=0) return false
if ( i<0 && j<0 ) return true;
if( j>=0 && i<0) {
    for( int k=0; k<=j; k++){
        if( p[k] != '*') {
    } return false;
    return true;
}
bool ans = false;
if (dp[i][j] != -1) return dp[i][j];
if (s[i] == p[j] || p[j] == '?')
    ans = solve (s, p, i-1, j-1);
else if ( p[j] == '*' ) {
    ans = solve (s, p, i, j-1, dp)
        || solve (s, p, i-1, j, dp);
else {
    ans = false;
}
return dp[i][j] = ans;
}
```

lec 140

GREEDY ALGORITHMS

→ we choose local optimum to get global optimum.

Q.1) N - meetings in one Room
return possible meetings.

→ 1 meeting at a time

room

N - meeting → start time: {1, 3, 0, 5, 8, 5}
end = {2, 4, 6, 7, 9, 9}

given → vector < pair < int, int > >
(start time, end time).

1) Sort all parts according to their end time.

→ i/p = ((1,2), (3,4), (0,6), (5,7), (8,9), (5,9))

sorted → ((1,2), (3,4), (0,6), (5,7), (8,9), (5,9))

initialise → st → 1
et → 2.

traverse left to right and check if current et < next st.

(1,2) ✓ et = 2
(3,4) ✓ (2 < 3) ✓ et = 4
(0,6) (4 < 0) ✗ et = 4
(5,7) (4 < 5) ✓ et = 5
(8,9) (5 < 8) ✓ et = 8
(5,9) (8 < 5) ✗ et = 8.

count = 4 → ans

code

```
static bool cmp ( pair <int, int> a,
pair <int, int> b ) {
    return a.second < b.second;
}

int solve ( int start[], int end[],
int n) {

    vector < pair < int, int > > v;
    for ( int i=0; i<n; i++){
      pair < int, int > p = make_pair(
      start [i], end [i]);
      v. push_back (p);
    }
    sort (v. begin(), v.end()), cmp);
    int count = 1;
    int ansEnd = v [0].second;
    for ( int i=1; i<n ; i++) {
      if ( v[i].first > ansEnd ) {
        count++;
        ansEnd = v[i].second;
      }
    }
    return count;
}
```

Q.2) Shop in Candy Store.

→ N - candies
→ buy 1 candy → get k candies for free (any type).
→ return min & max amount to buy all candies.

eg [3, 1, 2, 4] k = 2.

min → [3 1 2 4] = 3
F B B F

max = [3 1 2 4] = 7
B F F B

ans = 3 7

approach

1) Sort →

for min
[1 2 3 4]
↓ buy
take k free candies from end.

for max
[1 2 3 4]
take k free candies from start.
↓ Buy

Q.3) Check if it is possible to survive on island.

N → max food per day
S → days to survive
M → food per day to survive.

sunday → shop is closed.
curr day → monday.

return min days to buy food to survive for next S days if possible else ↳ (-1)

code

code keep sunday in mind.

```
int solve(int S, int N, int M){
    int sunday = S/7;
    int buyingDays = S - sunday;
    int totalFood = S*M;
    int ans = 0;
    if(totalFood % N == 0){
        ans = totalFood/N;
    }else{
        ans = totalFood/N + 1;
    }
    if(ans <= buyingDays)
        return ans;
    else
        return -1;
}
```

Q.4) Reverse words in given string.

i/p → I . am . Aadi

o/p → Aadi . am . I

```
string solve(string s){
    string ans = "";
    string temp = "";
    for(int i = s.length()-1;
        i<=0; i--){
        if(s[i] == '.'){
            reverse(temp.begin(),
                    temp.end());
            ans = ans + temp;
            temp = "";
            ans.push_back('.');
        }
        else{
            temp.push_back(s[i]);
        }
    }
    reverse(temp.begin(), temp.end());
    ans = ans + temp;
    return ans;
}
```

Q.5) Chocolate Distribution

↳ M students

↳ N packets

exactly one packet to each student

(maxi - mini) → should be minimum

approach 1) sort chocolate array.

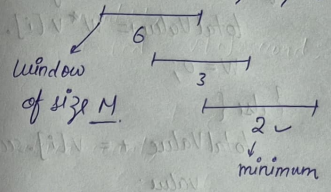
2) create a window of size M

3) store mini = (w[j] - w[i])

4) return mini.

M=3.

choco = {1, 5, 7, 8, 9}



distribute 3 student as 7, 8, 9

Q.6) Minimum Cost of Ropes.

i/p → 'n' ropes are given

arr = [4, 1, 3, 6]

To join two ropes i & j

cost = arr[i] + arr[j]

→ Join small ropes first

return min^m cost to join all ropes.

code → using min heap

```
int solve(vector<int> arr,
          int n){
    priority_queue<int,
        vector<int>, greater<int>>
        pq;
    for(int i=0; i<n; i++){
        pq.push(arr[i]);
    }
    int cost = 0;
    while(pq.size() > 1){
        int first = pq.top();
        int second =
            pq.pop();
        int second = pq.top();
            pq.pop();
        int length = first + second;
        cost = cost + length;
        pq.push(length);
    }
    return cost;
}
```

Q.7) Huffman Encoding

string s → string of 'n' distinct char.

freq[] → freq of that char.

Q.8) Fractional Knapsack

N → items W → capacity of KS

wt → [ ]

val → [ ]

return max^m value which can be stored in knapsack.

---

approach → calculate per unit value for each item

$$v = \frac{val}{wt}$$

put highest 'v' first ✓

---

code

```
double solve (int W, Item arr[],
    int n) {
  vector < pair < int, Item>> v;
  for ( int i=0; i<n; i++){
    double perUnitValue = ( 1.0*
    arr[i].value ) / arr[i].weight;
    pair < double, Item> p = make_pair(
    perUnitValue , arr[i]);
    v.push_back(p);
  }
  sort ( v.begin(), v.end, cmp);
  int totalValue = 0;
  double
  for ( int i=0; i< n; i++) {
    if( v[i].second.weight > w){
      totalValue += W*v[i].first;
      W = 0;
    }else {
      totalValue += v[i].second.
          value;
      W = W - v[i].second.weight;
    }
  }
  return totalValue;
}
```

Numbering fault in lec 140

Q.9) Job Sequencing Problem

→ find no. of Jobs done with maximum profit.

ip

Job Id	deadline	Profit
1	4	20
2	1	10
3	1	40
4	1	30.

approach

↳ sort on basis of profit.
& complete Job with highest profit first.

code

```
bool cmp ( Job a, Job b) {
    return a.profit > b.profit;
}

vector <int> solve (Job arr[],
    int n) {
  sort ( arr, arr+n, cmp);
  int maxiDead = INT_MIN;
  for (int i=0; i<n; i++){
    maxiDead = max (
        maxiDead, arr[i].dead);
  }
  vector <int> schedule (maxiDead+1,
      -1);
  int count = 0;
  int maxProfit = 0;
  for ( int i=0; i<n; i++) {
    int currProfit = arr[i].profit;
    int currJobID = arr[i].id;
    int currDead = arr[i].dead;
  for ( int k = currDead; k>0;
      k--) {
    if ( schedule[k] == -1){
      maxProfit += profit;
      schedule [k] = currJobID;
      count++;
      break;
    }
  }
}
vector <int> ans;
ans.push_back (count);
ans.push_back (maxProfit);
return ans;
}
```

28 Aug 2026.